



北京邮电大学

Beijing University of Posts and Telecommunications

移动终端安全

安全漏洞验证及软件破解

题	目	Android 应用软件破解
姓	名	
学	号	
班	级	
指	导	
教	师	

Android 应用软件破解

实验目标

Android 应用软件破解：对以下目标软件进行破解，编写破解报告，并提交破解后软件或注册机。

(1) 软件破解：使用 ApkTool 及 Dex2jar 逆向 crackme.apk，寻找到爆破点，修改注册逻辑代码，使注册机制失效，使用 ApkTool 重新打包生成 apk，要求新的 apk 对任意字符成功注册。（可使用工具：ApkTool、BakSmali 、dex2jar、jd-gui）。

(2) 注册机开发：逆向示例程序 crackme2.apk，根据 dex2jar 和 JD-GUI 工具，查看 java 源码，分析注册机制，写出注册机程序。

一、软件破解

软件破解部分的核心思路是：先将 APK 还原为可分析的目录结构，再定位注册判断逻辑，最后通过修改关键跳转或返回值改变程序执行路径。由于目标程序的验证逻辑在客户端本地完成，因此只要找到判断成功和失败的分支位置，就可以通过修改 Smali 指令使程序始终进入成功流程。

使用 ApkTool 及 Dex2jar 逆向 crackme.apk，寻找到爆破点，修改注册逻辑代码，使注册机制失效，使用 ApkTool 重新打包生成 apk，要求新的 apk 对任意字符成功注册。（可使用工具：ApkTool、BakSmali 、dex2jar、jd-gui）。

1.使用 ApkTool 反编译 APK

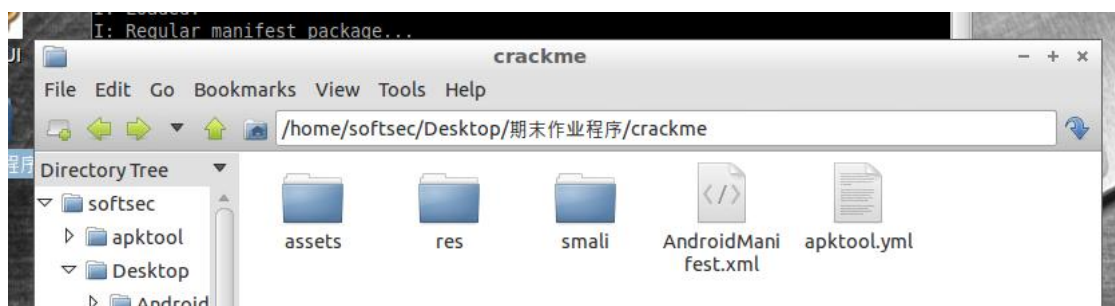
ApkTool 主要用于反编译 APK 中的资源文件和 Smali 代码。反编译后可以看到 AndroidManifest.xml、res 资源目录以及 smali 目录，这些文件能够帮助我们分析界面入口、权限申请、Activity 调用关系和具体的字节码逻辑。

这一步的作用相当于把 APK 从安装包形式拆解成便于阅读和修改的工程目录，为后续搜索注册关键字、定位校验函数和重新打包做准备。

```
cd /home/softsec/Desktop/期末作业程序
apktool d crackme.apk
```

```
softsec@softsec: ~/Desktop/期末作业程序
File Edit Tabs Help
softsec@softsec:~$ cd /home/softsec/Desktop/期末作业程序
softsec@softsec:~/Desktop/期末作业程序$ apktool d crackme.apk
I: Baksmaling...
I: Loading resource table...
I: Loaded.
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/softsec/apktool/framework/1.apk
I: Loaded.
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Done.
I: Copying assets and libs...
```

接下来我们可以得到反编译出来的文件夹



2.为了使用 JD-GUI 查看具体代码，我们使用 dex2jar 转换 dex 到 jar

dex2jar 的作用是将 APK 中的 classes.dex 转换为 jar 文件，再配合 JD-GUI 查看近似 Java 源码。虽然反编译结果不一定完全准确，但它可以快速帮助我们理解类名、方法名和主要控制流程。

在实际分析中，Java 反编译代码适合用于初步定位，真正修改程序行为时仍然需要回到 Smali 层，因为重新打包时修改的是反编译目录中的 Smali 指令。

```
d2j-dex2jar crackme.apk
```

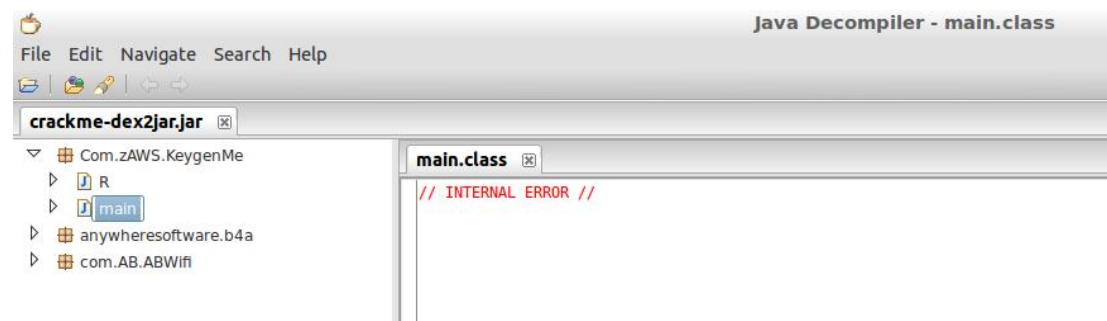
```
softsec@softsec:~/Desktop/期末作业程序$ d2j-dex2jar crackme.apk
dex2jar crackme.apk -> crackme-dex2jar.jar
softsec@softsec:~/Desktop/期末作业程序$
```



```
896
897     invoke-virtual {v0, p0}, Lanywheresoftware/b4a/objects/ImageViewWrapper;->setBitmap(Landroid/graphics/Bitmap;)V
898
899     .line 366
900     ✨ const-string v0, "Registered successfully\nPlease write Tutorial How Found key?!"
901
902     invoke-static {v0, v9}, Lanywheresoftware/b4a/keywords/Common;->ToastMessageShow(Ljava/lang/String;Z)V
903
```

在 JD-GUI 分析代码，发现 main 无法查看。这里说明目标程序可能存在混

淆、反编译兼容性问题，或者部分代码结构导致 JD-GUI 无法正确还原。遇到这种情况不能只依赖 Java 伪代码，需要结合 Smali 文件继续分析，避免因为反编译工具显示异常而误判程序逻辑。



3.使用 VS Code 直接打开 main.smali

Smali 是 Dalvik 字节码的文本表示形式，虽然可读性不如 Java 代码，但控制流和寄存器操作更接近程序真实执行过程。通过搜索 register、success、fail 等字符串，可以先定位界面提示文本，再沿着条件跳转语句向上反推校验逻辑。

搜索 register，查询注册成功状态。找到注册成功的输出后，从下往上看跳转，成功反推到检验部分可以看到，他是根据 check_code()的返回值决定跳转的，失败跳转到下面的 cond_4 对应的地方。

从结果提示向上反推是一种比较有效的逆向方法。因为注册成功或失败通常会调用 Toast、Dialog 或 TextView 显示固定字符串，先找到这些字符串所在位置，再观察 if-eqz、if-nez、goto 等跳转指令，就可以判断哪个分支对应成功，哪个分支对应失败。

这里的关键点是 check_code() 的返回值直接决定注册流程。如果返回值代表校验通过，程序会继续执行成功分支；如果返回值表示失败，就会跳转到 cond_4 对应的失败处理。因此修改返回值或跳转条件都可以达到绕过校验的效果。

```

872
873 * sput-object v0, LCom/zAWS/KeygenMe/main;-->_mac_address:Ljava/lang/String;
874
875 .line 362
876 invoke-static {}, LCom/zAWS/KeygenMe/main;-->_check_code()Z
877
878 move-result v0
879
880 if-eqz v0, :cond_4
881
882 .line 364
883 sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;
884
885 iget-object v0, v0, LCom/zAWS/KeygenMe/main;-->_imagev:Lanywheresoftware/b4a/objects/ImageViewWrapper;
886
887 sget-object v1, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;
888
889 iget-object v1, v1, LCom/zAWS/KeygenMe/main;-->_bmpul:Lanywheresoftware/b4a/objects/drawable/CanvasWrapper$BitmapWrapper;
890
891 invoke-virtual {v1, Lanywheresoftware/b4a/objects/drawable/CanvasWrapper$BitmapWrapper;-->getObject()Ljava/lang/Object;
892
893 move-result-object p0
894
895 check-cast p0, Landroid/graphics/Bitmap;
896
897 invoke-virtual {v0, p0, Lanywheresoftware/b4a/objects/ImageViewWrapper;-->setBitmap(Landroid/graphics/Bitmap;)V
898
899 .line 366
900 const-string v0, "Registered successfully\nPlease write Tutorial How Found key?!"
901
902 invoke-static {v0, v9}, Lanywheresoftware/b4a/keywords/Common;-->ToastMessageShow(Ljava/lang/String;)V
903
904 .line 374
905 :goto_2
906 sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

```

本实验选择强制修改返回值，是因为这种方式改动范围小，风险也比较低。相比删除整段校验代码，直接让关键寄存器保持“校验成功”的取值，不容易破坏后续界面更新、变量传递和方法调用流程。

```

875 .line 362
876 const/4 v0, 0x1
877
878 .line 364
879 sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

```

4.使用 apktool 重新打包，签名

Smali 修改完成后，需要使用 ApkTool 将目录重新编译为 APK。由于重新打包会改变原始 APK 的签名信息，系统不会再认为它是原开发者发布的安装包，所以还必须重新签名后才能正常安装到 Android 设备或模拟器中。

签名过程保证 APK 的完整性和安装合法性。实验中统一设置密码为 123456，便于后续重复打包和测试。安装前如果设备中已经存在同包名旧版本，通常需要先卸载旧版本，避免签名不一致导致安装失败。

```
apktool b crackme
```

```

softsec@softsec:~/Desktop/期末作业程序$ ls
crackme crackme2.apk crackme.apk crackme-dex2jar.jar
softsec@softsec:~/Desktop/期末作业程序$ apktool b crackme
I: Checking whether sources has changed...
I: Smaling...
I: Checking whether resources has changed...
I: Building resources...
I: Building apk file...
softsec@softsec:~/Desktop/期末作业程序$ █

```

```
keytool -genkey -v -keystore my-release-key.keystore -alias alias_name -keyalg
RSA -keysize 2048 -validity 10000
```

```

softsec@softsec:~/Desktop/期末作业程序$ keytool -genkey -v -keystore my-release-key.keystore -alias alias_n
ame -keyalg RSA -keysize 2048 -validity 10000
Enter keystore password:
Re-enter new password:
What is your first and last name?
[Unknown]: 1
What is the name of your organizational unit?
[Unknown]: 1
What is the name of your organization?
[Unknown]: 1
What is the name of your City or Locality?
[Unknown]: 1
What is the name of your State or Province?
[Unknown]: 1
What is the two-letter country code for this unit?
[Unknown]: 1
Is CN=1, OU=1, O=1, L=1, ST=1, C=1 correct?
[no]: yes

Generating 2,048 bit RSA key pair and self-signed certificate (SHA1withRSA) with a validity of 10,000 days
for: CN=1, OU=1, O=1, L=1, ST=1, C=1
Enter key password for <alias_name>
(RETURN if same as keystore password):
Re-enter new password:
[Storing my-release-key.keystore]
softsec@softsec:~/Desktop/期末作业程序$ █

```

```
jarsigner -verbose -keystore my-release-key.keystore crackme/dist/crackme.apk
alias_name
```

```

[Storing my-release-key.keystore]
softsec@softsec:~/Desktop/期末作业程序$ jarsigner -verbose -keystore my-release-key.keystore crackme/dist/c
rackme.apk alias_name
Enter Passphrase for keystore:
adding: META-INF/MANIFEST.MF
adding: META-INF/ALIAS_NA.SF
adding: META-INF/ALIAS_NA.RSA
signing: assets/1.bal
signing: assets/keys64.png
signing: assets/lock.png
signing: assets/unlock.png
signing: res/drawable/icon.png
signing: AndroidManifest.xml
signing: classes.dex
signing: resources.arsc
softsec@softsec:~/Desktop/期末作业程序$ █

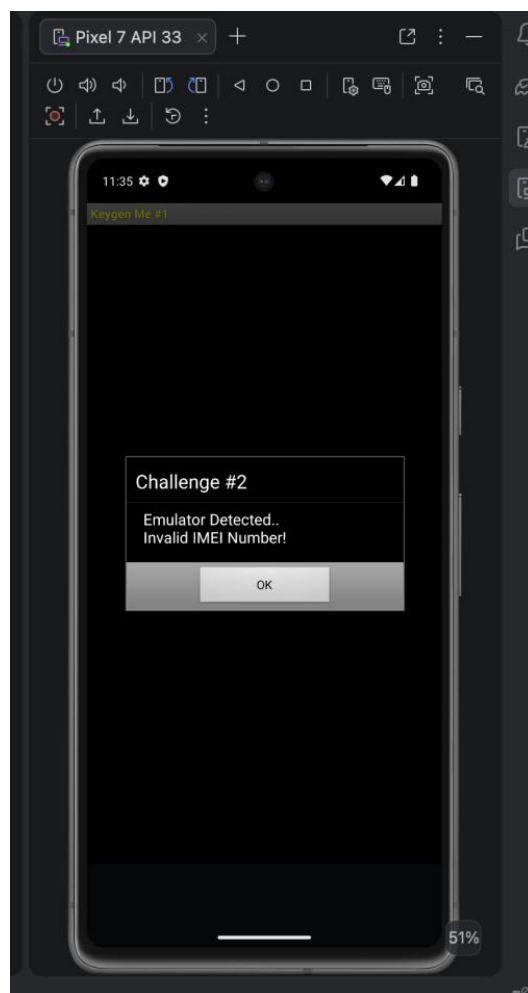
```

5.运行时发现问题，发现有检测 IMEI 码的部分，由于使用的是虚拟机，返回错误。

第一次绕过注册逻辑后程序仍然无法正常运行，说明目标软件除了注册码判断外，还加入了运行环境检测。由于模拟器获取到的 IMEI、MAC、Wi-Fi 信息

等可能为空或呈现典型模拟器特征，程序会据此判断当前环境不可信并中断流程。

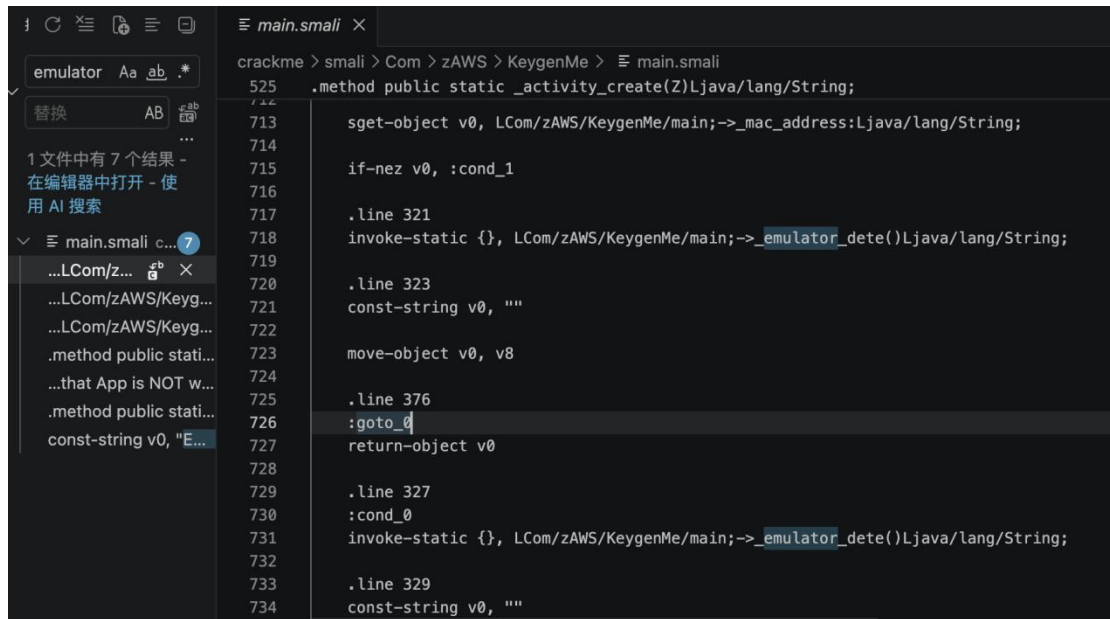
这类检测属于常见的反调试或反模拟器策略，目的在于提高逆向分析难度。解决思路仍然是定位检测函数，分析其返回值和异常处理路径，然后让程序跳过检测失败分支。



6.使用 VS Code 打开，搜索 emulator，一个个查看，很容易发现检测逻辑

搜索 emulator 能够快速找到与模拟器识别相关的字符串、方法名或分支逻辑。后续修改的原则是尽量保留程序原有流程，只改变检测结果，使程序认为当前环境满足真实设备条件。

下面几处修改分别处理 Wi-Fi 信息加载失败、MAC 地址为空、IMEI 无效、对象空指针和模拟器检测函数返回值等问题。多个位置一起修改，是因为环境检测往往不是单一判断，而是由多层条件共同构成。



(1) 修改位置 1: ABWifi 获取失败时的处理

该位置主要影响 Wi-Fi 信息是否成功加载。模拟器中 Wi-Fi 相关接口返回值可能不完整, 原逻辑会把这种情况当作异常环境处理。将返回结果调整为通过, 可以避免程序因为 ABWifi 加载失败直接进入错误分支。

项	内容
位置	_activity_create 方法内 .line 315
原始代码	ABLoadWifi(BA) 返回值存入 v0, 后续条件跳转判断: 若 false 则进入错误处理分支 → _emulator_dete() 弹窗 "WI-FI MAC Address required!" → Finish() 退出
问题	模拟器无真实 Wi-Fi 硬件, ABLoadWifi 返回 false, 触发退出
修改方式	将返回值后的条件跳转替换为 const/4 v0, 0x1, 强制覆盖为 true, 跳过错误处理分支

我们可以看到修改之前的程序

```

invoke-virtual {v0, v1, v2}, Lanywheresoftware/b4a/objects/drawable/CanvasWrapper$BitmapWrapper;-->Initialize
.line 313
sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

iget-object v0, v0, LCom/zAWS/KeygenMe/main;-->_mywifi:Lcom/AB/ABWifi/ABWifi;

sget-object v1, LCom/zAWS/KeygenMe/main;-->processBA:Lanywheresoftware/b4a/BA;

invoke-virtual {v0, v1}, Lcom/AB/ABWifi/ABWifi;-->ABLoadWifi(Lanywheresoftware/b4a/BA;)Z

move-result v0

.line 315
if-ne v0, v9, :cond_0

.line 317
sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

```


改为 (让 ABWifi 的 ABLoadWifi 永远返回 true, 跳过:cond_0 直接进入真实设备逻辑)

```
move-result v0

.line 315
const/4 v0, 0x1

.line 317
sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

iget-object v0, v0, LCom/zAWS/KeygenMe/main;-->_mywifi:Lcom/AB/ABWifi/ABWifi;

invoke-virtual {v0}, Lcom/AB/ABWifi/ABWifi;-->ABGetCurrentWifiInfo()Lcom/AB/ABWifi/ABWifi$ABWifiInfo;
```

(2) 修改位置 2 —— MAC 地址判空条件绕过

MAC 地址判空用于判断设备网络信息是否完整。模拟器或高版本 Android 系统中，应用可能无法正常获取真实 MAC 地址，因此这里将判空失败分支绕过，让程序继续执行后续逻辑。

项	内容
位置	_activity_create 方法内 .line 319
原始代码	_mac_address 获取后判空，若为空字符串则条件跳转至 :cond_0 → _emulator_dete() 弹窗退出; 若非空则 goto :cond_1 进入 IMEI 获取流程
问题	模拟器上 Wi-Fi 信息对象可能返回 null 或 MAC 为空字符串，触发退出
修改方式	将条件跳转改为无条件 goto :cond_1，直接跳过 MAC 有效性检查

我们可以看到之前的程序

```
.line 319
sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

sget-object v0, LCom/zAWS/KeygenMe/main;-->_mac_address:Ljava/lang/String;

if-nez v0, :cond_1

.line 321
invoke-static {}, LCom/zAWS/KeygenMe/main;-->_emulator_dete()Ljava/lang/String;
```

改为 (强制跳过判空，进入后续流程，即使 MAC 地址为 null)

```

710      .line 319
711      sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;
712
713      sget-object v0, LCom/zAWS/KeygenMe/main;-->_mac_address:Ljava/lang/String;
714
715      goto :cond_1
716

```

(3) 修改位置 3 —— IMEI 合法性检测绕过

IMEI 是程序识别设备的重要依据，但模拟器环境下 IMEI 可能为空、固定或格式异常。绕过该处合法性检测后，程序不会再因为 v3 的取值异常调用模拟器检测函数，从而减少运行时拦截。

项	内容
位置	_activity_create 方法内 .line 349
原始代码	遍历 IMEI 字符串每个字符以十六进制解析并累加至 v3，循环结束后判 v3 值：若为模拟器特征值则跳转至 _emulator_detel() 弹窗 "Emulator Detected.. Invalid IMEI Number!" → Finish() 退出；若正常则 goto :cond_3 继续后续
问题	模拟器 IMEI 为 0000000000000000，累加结果 v3 = 0，被判定为非法，触发退出
修改方式	将循环后的条件跳转改为无条件 goto :cond_3，绕过 IMEI 检测

我们可以看到之前的程序

```

779
780      if-lez v4, :cond_2
781
782      .line 349
783      if-nez v3, :cond_3
784
785      .line 351
786      ✨ invoke-static {}, LCom/zAWS/KeygenMe/main;-->_emulator_detel()Ljava/lang/String;
787
788      .line 353

```

改为（绕过 v3 == 0 时执行 _emulator_detel() 的逻辑）

```

779
780      if-lez v4, :cond_2
781
782      .line 349
783      goto :cond_3
784
785      .line 351
786      ✨ invoke-static {}, LCom/zAWS/KeygenMe/main;-->_emulator_detel()Ljava/lang/String;
787
788      .line 353

```

(4) 修改位置 4 —— Wi-Fi 信息对象空指针保护

这里的修改主要是为了提高运行稳定性。如果 Wi-Fi 信息对象为空而程序

继续访问其成员，就可能触发空指针异常。通过调整分支或返回值，可以避免程序在模拟器环境中直接崩溃。

项	内容
位置	_activity_create 方法，ABGetCurrentWifiInfo() 调用后
原始代码	ABGetCurrentWifiInfo() 返回值直接访问 .MacAddress 字段, 无 null 检查
问题	模拟器上该方法返回 null，执行 null.MacAddress 触发 NullPointerException，程序闪退。该崩溃发生在修改位置 2 的跳转逻辑之前，尚未走到判空分支即已崩溃
修改方式	在 move-result-object v0 后插入 if-nez v0 判空：若 null 则赋值默认 MAC "000000000000" 并跳转至 :after_mac；若非 null 则正常读取 .MacAddress

我们可以看到之前的程序

```
invoke-virtual {v0}, Lcom/AB/ABWifi/ABWifi;->ABGetCurrentWifiInfo()Lcom/AB/ABWifi/ABWifi$ABWifiInfo;
move-result-object v0

iget-object v0, v0, Lcom/AB/ABWifi/ABWifi$ABWifiInfo;->MacAddress:Ljava/lang/String;
sput-object v0, LCom/zAWS/KeygenMe/main;->_mac_address:Ljava/lang/String;
```

修改

```
702 invoke-virtual {v0}, Lcom/AB/ABWifi/ABWifi;->ABGetCurrentWifiInfo()Lcom/AB/ABWifi/ABWifi$ABWifiInfo;
703
704 move-result-object v0
705
706 if-nez v0, :wifi_info_ok
707
708 const-string v0, "000000000000"
709
710 sput-object v0, LCom/zAWS/KeygenMe/main;->_mac_address:Ljava/lang/String;
711
712 goto :after_mac
713
714 :wifi_info_ok
715 iget-object v0, v0, Lcom/AB/ABWifi/ABWifi$ABWifiInfo;->MacAddress:Ljava/lang/String;
716
717 sput-object v0, LCom/zAWS/KeygenMe/main;->_mac_address:Ljava/lang/String;
718
719 :after_mac
720
```

(5) 修改位置 5 —— IMEI 获取空指针保护

IMEI 获取接口同样可能返回空对象或空字符串。加入绕过或保护后，即使设备标识无法正常读取，程序也能继续执行到注册流程，便于验证前面修改后的破解效果。

项	内容
位置	_activity_create 方法，GetDeviceId() 调用后
原始代码	PhoneId.GetDeviceId() 返回值直接存入 _imei，随后立即调用 _imei.length()，无 null 检查
问题	Android 10+ 上 getDeviceId() 对非系统应用返回 null，执行 null.length() 触发 NullPointerException，程序闪退。该崩溃发生在修改位置 3 的跳转逻辑之前，尚未走到 v3 检测即已崩溃
修改方式	在 sput-object v0, _imei 后插入 if-nez v0 判空：若 null 则赋值默认 IMEI "358240051111110" 覆写 _imei

我们可以看到之前的程序

```
sput-object v0, LCom/zAWS/KeygenMe/main;-->_imei:Ljava/lang/String;

.line 340
sget-object v0, LCom/zAWS/KeygenMe/main;-->mostCurrent:LCom/zAWS/KeygenMe/main;

sget-object v0, LCom/zAWS/KeygenMe/main;-->_imei:Ljava/lang/String;

invoke-virtual {v0}, Ljava/lang/String;-->length()I
```

修改后

```
sput-object v0, LCom/zAWS/KeygenMe/main;-->_imei:Ljava/lang/String;

if-nez v0, :imei_ok

const-string v0, "358240051111110"

sput-object v0, LCom/zAWS/KeygenMe/main;-->_imei:Ljava/lang/String;

:imei_ok
.line 340
```

(6) 修改位置 6 ——模拟器检测函数体清空

最后清空或改写模拟器检测函数体，相当于从根源上让检测函数不再报告异常环境。这样可以避免前面某些分支遗漏时再次触发检测失败，提高破解 APK 的运行成功率。

项	内容
位置	_emulator_dete() & _emulator_dete1()
原始代码	两函数分别弹出 "WI-FI MAC Address required!" / "Emulator Detected.. Invalid IMEI Number!" 对话框后调用 ActivityWrapper.Finish() 退出程序
问题	上述五处修改后正常路径已不会调用这两个函数，但为保险起见，防止遗漏的代码路径仍然触发弹窗退出
修改	删除两函数体内全部逻辑（Msgbox + Finish），改为 .locals 1 + 直

项	内容
方式	接 return-object v0 (空字符串)

修改前

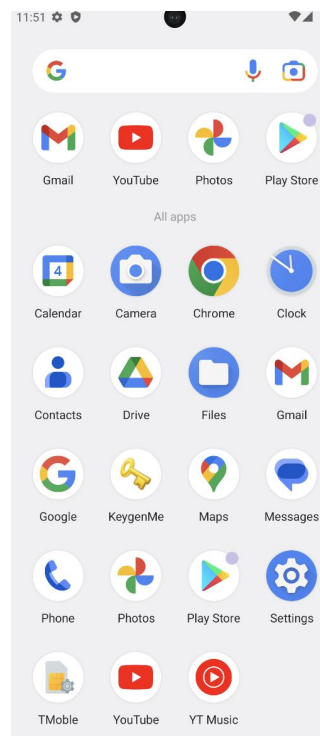
```
.method public static _emulator_dete1(Ljava/lang/String;
    .locals 3
    const-string v0, "Emulator Detected..\nInvalid IMEI Number!"
    const-string v1, "Challenge #2"
    # ... Msgbox ... Finish ...
    return-object v0
.end method
```

修改后

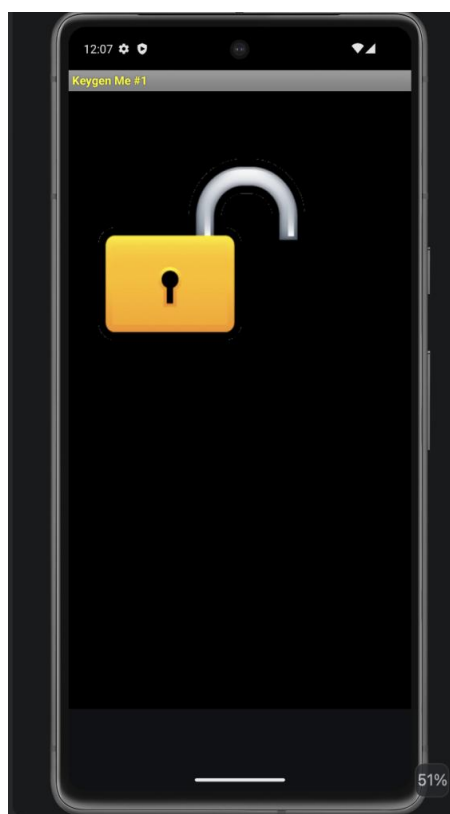
```
.method public static _emulator_dete1(Ljava/lang/String;
    .locals 1
    const-string v0, ""
    return-object v0
.end method
```

7.最后成功安装启动

经过注册判断和模拟器检测两部分修改后, 重新打包签名的 APK 可以正常安装并启动。此时再输入任意字符进行注册, 程序会按照修改后的控制流进入成功分支, 说明关键校验逻辑已经被绕过。



运行结果:



二、注册机开发

注册机开发与直接修改 APK 的思路不同。前者不是改变目标程序本身，而是还原其注册码生成算法，然后在外部根据相同规则计算出正确注册码。该部分实验重点在于理解目标程序如何绑定设备 ID、如何加工哈希值，以及如何将算法移植为独立程序。

逆向示例程序 crackme2.apk，根据 dex2jar 和 JD-GUI 工具，查看 java 源码，分析注册机制，写出注册机程序。

1.我们前面的准备步骤已经把 crackme2.apk 提取出来了，现在我们直接快进到用 JD-GUI 查看代码即可。

分析 crackme2.apk 时,首先通过 JD-GUI 查看主要 Activity 和注册验证函数，寻找用户输入注册码与程序内部计算值之间的比较位置。只要定位到比较函数，就可以继续追踪注册码的生成来源。


```

public void onClick(View paramView) {
    switch (paramView.getId()) {
        default:
            return;
        case 2131034116:
            break;
    }
    EditText editText = (EditText)findViewById(2131034115);
    if (validateSerial(editText.getText().toString()) == 0)
        Toast.makeText((Context)this, "Invalid code.\nTry again.", 1).show();
    Toast.makeText((Context)this, "Code is valid!", 1).show();
    editText.setVisibility(4);
    ((Button)findViewById(2131034116)).setVisibility(4);
    ((TextView)findViewById(2131034114)).setText("Code Accepted :D");
}

public void onCreate(Bundle paramBundle) {
    super.onCreate(paramBundle);
    setContentView(2130903040);
    ((Button)findViewById(2131034116)).setOnClickListener(this);
}
}

```

validateSerial() 是整个注册判断的入口函数，它通常会接收用户输入的序列号，然后调用内部生成函数得到理论正确值，最后通过字符串比较判断是否注册成功。因此该函数是注册机分析中最关键的位置。

```

private int validateSerial(String paramString) {
    try {
        boolean bool = generateIDHash().equals(paramString);
        if (bool)
            return 1;
    } catch (Exception exception) {
        exception.printStackTrace();
    }
    return 0;
}

```

进一步分析，发现是将输入值与一个函数值比较，继续分析函数 generateIDHash()

generateIDHash() 负责根据设备信息生成注册码。这里可以看出软件采用了“设备绑定”的授权方式，也就是同一个注册码只对特定设备有效，换到其他设备后由于 IMEI 不同，计算出的注册码也会变化。

```

public void onClick(View paramView) {
    switch (paramView.getId()) {
        default:
            return;
        case 2131034116:
            break;
    }
    EditText editText = (EditText)findViewById(2131034115);
    if (validateSerial(editText.getText().toString()) == 0)
        Toast.makeText((Context)this, "Invalid code.\nTry again.", 1).show();
    Toast.makeText((Context)this, "Code is valid!", 1).show();
    editText.setVisibility(4);
    ((Button)findViewById(2131034116)).setVisibility(4);
    ((TextView)findViewById(2131034114)).setText("Code Accepted :D");
}

public void onCreate(Bundle paramBundle) {
    super.onCreate(paramBundle);
    setContentView(2130903040);
    ((Button)findViewById(2131034116)).setOnClickListener(this);
}
}

```

发现两个函数组合实现的功能大致是获取本机设备 ID (IMEI) 。

IMEI 获取完成后，程序并不是直接使用原始设备号，而是继续进行 MD5 摘要和字符串加工。这样做可以隐藏真实设备 ID，同时让注册码长度更短、更像序列号。使用 MD5 哈希并加工处理生成注册码。

核心流程就很显而易见了：根据代码逻辑，可以把注册验证拆分成“取设备标识、计算哈希、截取或变换字符串、与用户输入比较”四个阶段。注册机只要严格复现这几个阶段，就能生成目标程序认可的注册码。1.获取本机设备 ID (IMEI) 。2.使用 MD5 哈希并加工处理生成注册码。3.用户在界面中输入注册码，程序比对是否匹配设备计算结果。4.匹配成功提示通过，否则提示错误。注册机程序只需要根据该程序逻辑生成注册码并输出就行了

编写注册机时需要特别注意两点：一是哈希算法、大小写、编码方式要和原程序保持一致；二是字符串加工顺序不能出错，否则即使使用同一个 IMEI，最终生成的注册码也无法通过验证。编写时发现他这个代码逻辑有问题，应该是加了反编译处理，无法执行，还得结合 class 代码进行反编译。

2.结合 class 代码，发现用 JD-GUI 查看的出来的部分果然有逻辑上的错误，实际上的加工应该是全部都经过了一遍，但图中的却在第一次循环就退出了，并且还没有返回值。

这一步体现了反编译分析中必须交叉验证的重要性。JD-GUI 生成的是近似 Java 代码，遇到混淆、异常控制流或反编译器识别失败时，可能会出现循环提前结束、返回值缺失等不符合真实执行逻辑的情况。

因此需要结合 class/字节码或 Smali 指令确认真实逻辑。确认循环完整执行后，才能准确还原注册码生成过程，避免按照错误的伪代码写出无法使用的注册机。

还原后的逻辑表明，程序会对 MD5 结果的多个字符依次进行加工，而不是只处理第一个字符。注册码的每一位都来自完整循环结果，因此循环次数、下标范围和拼接顺序都会影响最终结果。

J KeygenCrackme2.java 1 X

J KeygenCrackme2.java > KeygenCrackme2 > generateSerial(String)

```
1  import java.math.BigInteger;
2  import java.security.MessageDigest;
3
4  /**
5   * crackme2.apk 注册机
6   * 包名: com.lohan.crackme0a
7   * 算法: MD5(IMEI) -> 相邻字节 XOR -> hex -> 前15位
8   */
9  public class KeygenCrackme2 {
10
11     public static String generateSerial(String imei) throws Exception {
12         // 1. 计算 IMEI 的 MD5
13         MessageDigest md = MessageDigest.getInstance("MD5");
14         md.update(imei.getBytes(), offset: 0, imei.length());
15         byte[] digest = md.digest();
16
17         // 2. Pair-XOR 变换: digest[i] ^ digest[i+1]
18         // 最后一位如果落单则 XOR 0
19         byte[] transform = new byte[digest.length];
20         int tPos = 0;
21         for (int dPos = 0; dPos < digest.length; dPos += 2) {
22             int nextPos = (dPos == digest.length - 1) ? 0 : dPos + 1;
23             transform[tPos] = (byte) (digest[dPos] ^ digest[nextPos]);
24             tPos++;
25         }
26
27         // 3. 转十六进制字符串
28         // new BigInteger(1, bytes) 保证是正数
29         byte[] result = new byte[tPos];
30         System.arraycopy(transform, srcPos: 0, result, destPos: 0, tPos);
31         String hex = new BigInteger(signum: 1, result).toString(radix: 16);
32
33         // 4. 取前 15 位
34         return hex.substring(beginIndex: 0, endIndex: 15);
35     }
36
37     public static void main(String[] args) throws Exception {
38         // 测试模拟器默认 IMEI
39         String imei1 = "358240051111110";
40         System.out.println("IMEI: " + imei1);
41         System.out.println("Serial: " + generateSerial(imei1));
42
43         System.out.println();
44
45         // 测试模拟器特征 IMEI
46         String imei2 = "0000000000000000";
47         System.out.println("IMEI: " + imei2);
48         System.out.println("Serial: " + generateSerial(imei2));
49
50         // 如果命令行提供了 IMEI 参数则使用
51         if (args.length > 0) {
52             System.out.println();
53             System.out.println("IMEI: " + args[0]);
54             System.out.println("Serial: " + generateSerial(args[0]));
55         }
56     }
57 }
58
```

3.根据逻辑写出注册机程序，生成他要的输入码，得到 46da69e0a4a5cab。

注册机程序运行后输出 46da69e0a4a5cab，说明外部程序已经成功复现目标 APK 的注册码生成算法。将该结果输入原程序后，如果能够通过验证，就可以证明算法分析和代码还原是正确的。

```
(base) lihaolun@lihaolundeMacBook-Air 目标程序 % /usr/bin/env /opt/homebrew/Cellar/openjdk@17/17.0.19/libexec/openjdk.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/lihaolun/Library/Application\ Support/Code/User/workspaceStorage/8686b726e9bee14354d8c841ce7ef59c/redhat.java/jdt_ws/jdt.ls-java-project/bin KeygenCrackme2
IMEI: 35824005111110
Serial: 46da69e0a4a5cab

IMEI: 000000000000000
Serial: d67bb44ac8fe21a
```

将这个注册码输入程序验证





结果通过

最终验证通过，说明 crackme2.apk 的注册机制完全依赖客户端本地算法，没有服务端二次校验。该实验说明，单纯把注册码生成和验证逻辑放在客户端并不安全，攻击者可以通过逆向工程还原算法并编写注册机。

实验总结：本次实验分别采用了修改程序控制流和还原注册码算法两种方式完成破解。前者说明本地校验分支可以被直接绕过，后者说明本地授权算法可以被复现。实际软件开发中应尽量减少敏感逻辑在客户端暴露，并结合服务端校验、代码混淆、完整性检测和运行环境检测等多种方式提升安全性。